CUEGATE

A fail-closed subtitle verification and repair tool

GATE: PASS

Subtitles that fit the frame, or don't ship.

CONSTRUCTED illustration, not a real measurement

OWNER
Deepak Kumaran T S

COURSE
INFO 7375 · Fall 2026

DATE
2026-10-14

STATUS
PLANNING · NOTHING RUNS YET

Subtitles that fit the frame, or don't ship.

> PLANNING DOCUMENT. Written 2026-10-14 by Deepak Kumaran T S for INFO 7375. Nothing described here runs yet. Every capability is written in the future tense on purpose. Section 22 is the only list of things that exist.

Field	Value
Working title	CueGate
Owner	Deepak Kumaran T S
Course	INFO 7375 Prompt Engineering and Agentic AI, Fall 2026
Target role	Applied AI Engineer / AI Engineer
Primary reader	Subtitle QC lead at a localization vendor
Stack	Python 3.11, Claude API, PySceneDetect, FastAPI, React + Vite
Status	Planning. No code.

01 WHAT IT IS

CueGate will be a subtitle verification and repair tool. It will find every subtitle a viewer cannot read in time, fix the timing first, shorten the text only where timing cannot fix it, and refuse to ship any line it cannot prove fits.

When it is done, it will exist as four things:

- A Python package, `cuegate`, holding the rule engine, the repair loop and the meaning guard.
- A command-line tool: `cuegate check`, `cuegate fix`, `cuegate eval`.
- A local web app for reviewing a film's subtitles against its shot changes.
- An evaluation report on public open-movie subtitles, including a hand-labelled audit of how often the shortened lines changed meaning.

02 THE PROBLEM

Streaming platforms publish strict, numeric delivery rules for subtitle files. Netflix's timed text style guide sets limits on reading speed, line length, event duration, the gap between consecutive events, and how events must be timed around shot changes. Secondary guides report that files breaking these limits are flagged by automated QC before a human reviewer sees them, and that reading speed is one of the most common rejection reasons.

The rules are easy to check and hard to satisfy. When a line is too fast to read, the fix is either more screen time (which can collide with the next line or a cut) or fewer characters (which risks changing what the character said). Netflix's own template guidance tells subtitlers to avoid condensing unnecessarily and not to paraphrase. That is a precise, constrained editing problem, and it is still largely done by people, one line at a time.

03 WHAT ALREADY EXISTS (CHECKED 2026-10-14)

This section matters more than any other, because the pitch rubric penalises invented competitor claims.

> CORRECTION. An earlier draft of this plan relied on a vendor blog stating that no commercial tool automatically rewrites subtitle text to hit a reading-speed target. A check on 2026-10-14 found that this is no longer accurate. The claim has been dropped. The differentiation below is narrower and is stated only in terms of what CueGate does.

Tool	What its own documentation says it does	Source
Subtitle Edit (open source)	An AI Assistant that works on the current line: fix errors, fit reading speed, change tone. Selection by CPS, gaps and duration.	subtitleedit.github.io
Editsub	Flags high CPS, long lines, overlaps and tiny gaps. Offers a per-cue "Shorten with AI" action and bulk timing fixes.	editsub.com
EZTitles	Automated QC that flags reading speed, characters per line and timing errors; shot-change detection; reading-speed-aware segmentation.	eztitles.com
Subtitle AI	Checks every cue's reading speed (spaces excluded) against house rules; configurable line length and gaps.	subtitle.ai
Smartcat Subtitle Editor	Shows a CPS value per cue; a human adjusts timing, splits and merges.	help.smartcat.com

Where CueGate will differ. In the tools checked, AI shortening is an action a person triggers on one line, and detection is reported for a person to resolve. CueGate's difference will be three things, each of which can be checked in the repo:

1. **A fail-closed gate over the whole file.** Every repaired line is re-verified against the full rule set, including its neighbours and the shot changes, before it is accepted. A line that cannot be proven to fit is left unchanged and escalated with a reason. The output file will never contain a modified line that fails a rule.
2. **Cheapest fix first.** Timing is repaired deterministically before any text is touched. The model is called only for lines timing cannot fix, and is given an exact character budget.
3. **A published meaning-drift number.** CueGate will ship an evaluation that measures how often shortened lines changed meaning, against a hand-labelled audit, and will report the number whether it is good or bad. No published figure of this kind was found for the tools above; this must be re-checked before the film.

04 WHO OPENS IT, AND WHAT THEY DO IN THE FIRST SIXTY SECONDS

Primary reader: a subtitle QC lead or localization engineer at a vendor that delivers to streaming platforms.

First sixty seconds: drop in an English subtitle file and the film (or a cut list). Within a minute they see a table: every rule broken, by cue; every cue CueGate repaired and how (timing, line break, or text); every cue it refused to touch, with the reason. One click exports the repaired file and the report.

Secondary reader: a hiring manager for applied-AI roles, who opens the repo, runs `cuegate eval` on the bundled open-movie corpus, and reads the audit numbers.

05 DESIGN PRINCIPLES

1. **The counter decides, not the model.** All acceptance decisions are made by deterministic code. The model proposes; it never approves.
2. **Cheapest fix first.** Timing, then line breaking, then resegmentation, then condensing. Text is the last resort.
3. **Minimal intervention.** Among passing candidates, prefer the one that changes the least, following the guidance to avoid unnecessary condensing and paraphrase.
4. **Fail closed.** Uncertainty means escalation, never a silent change.
5. **Every rule cites its source.** Each rule carries the guideline section it implements and the date that section was read.
6. **Integer frames, never floats.** All timing math runs on integer frame numbers at a rational frame rate.
7. **Reproducible runs.** Pinned model ID, versioned prompts, a response cache, and the date of every model call recorded in the run log.

06 ARCHITECTURE



Module	Responsibility
`cuegate.io`	Parse SRT and WebVTT via pysubs2; normalise encoding, BOM, line endings; reject malformed files with line numbers.
`cuegate.timebase`	Convert milliseconds to integer frames at a rational fps (24, 25, 24000/1001, 30000/1001).
`cuegate.text`	Grapheme-accurate counting, formatting-tag stripping, line metrics under the profile's counting rule.
`cuegate.shots`	Shot detection with caching by video hash; cut-list import as a fallback.
`cuegate.rules`	One file per rule, each with an id, a source reference and a pure `check()`.
`cuegate.repair`	Timing solver, line breaker, resegmenter, condenser, selector, bounded loop.
`cuegate.guard`	Meaning invariants and the NLI entailment check.
`cuegate.llm`	Claude client with pinned model, retries, JSON-schema output, SQLite response cache, call log with dates.
`cuegate.eval`	Corpus loader, stress generator, baselines, metrics, human-audit tooling, report writer.
`cuegate.api`	FastAPI server for the web app.
`web/`	React + Vite review app.

07 DATA MODEL

All times are integer frames. Text is stored raw (with tags) and measured stripped.

```
@dataclass(frozen=True)
class FrameRate:
    num: int              # 24000
    den: int              # 1001

@dataclass(frozen=True)
class Cue:
    idx: int
    in_f: int             # first frame shown
    out_f: int            # first frame no longer shown
    raw: str              # original text, tags included
    lines: tuple[str, ...]
    flags: frozenset[str] # {"lyrics", "forced", "speaker_dash", "continues", ...}

@dataclass(frozen=True)
class Violation:
    rule_id: str          # "CPS_MAX"
    cue_idx: int
    measured: float       # 24.3
    limit: float          # 20.0
    source: str           # "Netflix TTSG, Timing, s.2 (read 2026-10-14)"

@dataclass
class CueResult:
    cue: Cue
    action: str           # "none" | "timing" | "line_break" | "resegment" | "condense" | "escalated"
    after: Cue | None
    checks: dict          # every rule result, invariant result and NLI score
    reason: str | None    # required when escalated
```

08 RULE ENGINE

Rules run as pure functions over the timeline, the cut list and a profile. Profiles are YAML so a vendor can add a house style without code.

```
# profiles/en-adult.yaml   (values to be confirmed against the primary source)
id: en-adult
source: "Netflix Timed Text Style Guide, English, read YYYY-MM-DD"
max_cps: 20              # secondary sources: 20 English adult, 17 English kids
cps_counting:
  include_spaces: true    # TO VERIFY. Some tools exclude spaces.
  include_punctuation: true
max_chars_per_line: 42
max_lines: 2
min_duration_s: 0.8333    # 5/6 of a second
max_duration_s: 7.0
min_gap_frames: 2
shot_change:
  snap_window_s: 0.5      # in-time snaps to a cut if speech starts within 0.5 s after it
  out_snap_window_s: 0.5  # out-time within 0.5 s before a cut extends to the cut
  gap_from_cut_frames: 2
linger_max_s: 0.5         # how far an out-time may extend past the source out-time
```

Rule id	Check	Notes
CPS_MAX	characters / duration <= max_cps	Counting convention comes from the profile.
CPL_MAX	every line <= max_chars_per_line	Measured in grapheme clusters, tags stripped.
LINES_MAX	line count <= max_lines	One line preferred when it fits.
DUR_MIN	duration >= min_duration	
DUR_MAX	duration <= max_duration	
GAP_MIN	next.in - out >= min_gap_frames	
OVERLAP	no two cues share a frame	Unless the profile allows overlapping speakers.
CUT_CROSS	a cue may not newly cross a shot change	Allowed only when the source cue already did (dialogue crosses the cut).
CUT_IN_SNAP	in-time within snap window after a cut should sit on the cut	Reported as a warning when the source never snapped.
CUT_OUT_SNAP	out-time within snap window before a cut should sit at cut minus gap	

Each violation carries the rule's source string, so a report line can always be traced back to the guideline it enforces.

09 THE REPAIR ALGORITHM

This is the core of the project. It runs per cue, in order, and stops at the first step that produces a passing result.

9.1 CLASSIFY

For each violating cue, compute what kind of fix could work. Duration and gap violations are timing problems. Line-length violations may be line-break problems. Reading-speed violations are timing problems if extra screen time exists, and text problems only if it does not.

9.2 TIMING SOLVER (DETERMINISTIC)

Find the widest legal window the cue may occupy without disturbing anything else.

```
def legal_window(i, tl, cuts, p, video_end):
    c = tl[i]
    prev_out = tl[i-1].out_f if i > 0 else 0
    next_in  = tl[i+1].in_f  if i + 1 < len(tl) else video_end

    # ceiling for the out-time
    ceil = next_in - p.min_gap
    ceil = min(ceil, c.in_f + p.max_dur)          # never longer than max duration
    ceil = min(ceil, c.out_f + p.linger_max)      # never far past the source out-time
    cut = first_cut_between(cuts, c.out_f, ceil)
    if cut is not None and not crosses_cut(c, cuts):
        ceil = cut - p.gap_from_cut                # land just before the cut

    # floor for the in-time: only moves earlier to snap onto a cut
    floor = c.in_f
    cut = last_cut_between(cuts, c.in_f - p.snap_window, c.in_f)
    if cut is not None and cut - prev_out >= p.min_gap:
        floor = cut

    return floor, max(ceil, c.out_f)
```

The source out-time is treated as an approximation of the end of speech, because v1 has no audio analysis. That is a stated limitation (section 16).

9.3 CHARACTER BUDGET

```
seconds = (ceil - floor) * fps.den / fps.num
budget  = math.floor(p.max_cps * seconds)          # under the profile's counting rule
budget  = min(budget, p.max_lines * p.max_chars_per_line)
```

If the current text already fits the budget, the timing fix alone resolves the cue. No model call is made.

9.4 LINE BREAKER (DYNAMIC PROGRAMMING)

After any text is chosen, lines are broken deterministically.

- If the whole text fits on one line, use one line.
- Otherwise enumerate every break point between tokens. A break is feasible only if both lines fit `max_chars_per_line`.
- Cost of a break = imbalance between line lengths + penalty for ending line one on an article, preposition, auxiliary or conjunction + penalty for splitting a name + bonus for breaking after punctuation + small bonus for a bottom-heavy shape.
- Pick the lowest-cost feasible break. Two-line enumeration is linear in the number of tokens.
- Never re-break across a speaker dash.

9.5 RESEGMENTATION

For cues over the maximum duration, split at the best sentence or clause boundary, with timing divided in proportion to characters and the minimum gap inserted. For neighbouring cues that are each below minimum duration and belong to the same speaker, try a merge, then re-run every rule on the merged result.

9.6 CONDENSING (THE PROMPT CONTRACT)

Only cues that steps 9.2 to 9.5 cannot fix reach the model. The call is a contract, not a chat.

Inputs sent:

- The cue text, wrapped in delimiters and labelled as data to be edited, never as instructions.
- Two cues of context before and after, labelled read-only.
- The exact character budget and the counting rule.
- A `must_keep` list computed deterministically: numbers, negations, names not established in earlier cues, the question mark, speaker dashes, formatting tags.
- The allowed editing strategies, taken from the guideline: deletion, contractions, removing repetitions, removing established names, reformulation as a last resort.

Output required (JSON schema, enforced and validated):

```
{
  "candidates": [
    {
      "text": "string",
      "strategy": ["deletion", "contraction"],
      "dropped": ["really", "I think"]
    }
  ]
}
```

Four candidates per call. Temperature 0. Pinned model. Every request and response is cached by a hash of (text, budget, context, prompt version, model) and logged with its date.

9.7 MEANING GUARD

Two layers, of different strength. The film will say which is which.

Layer 1: invariants (deterministic, strong for what they cover).

Invariant	Rule
Numbers	Every number in the source survives, with "three" and "3" normalised as equal.
Negation	Every negation token survives: not, n't, no, never, nothing, nobody, none, nor, neither, without, cannot.
Names	Every name survives unless it was already established in the previous cues (the guideline permits removing established names).
Question	A question remains a question.
Speakers	Speaker dash count is unchanged.
Tags	Formatting tags remain balanced and present.
Register	Profanity in the source is not removed.

Layer 2: NLI entailment (probabilistic, weaker). A local cross-encoder NLI model scores whether the source entails the candidate. A shorter line should say nothing the source did not say. This catches additions and contradictions. It does not catch harmless information loss, which condensing causes by design; meaning-critical loss is what layer 1 exists for. The threshold will be calibrated on the human audit, not guessed.

9.8 SELECTION

```
passing = [c for c in candidates
           if rules_pass(apply(c)) and invariants_pass(c) and nli(c) >= tau]
best = max(passing, key=lambda c: (retained_token_ratio(src, c), nli(c)))
```

Minimal change wins first; semantic score breaks ties.

9.9 THE BOUNDED LOOP AND ESCALATION

```
for round in range(3):
    cands = llm.condense(cue, context, budget, must_keep, feedback)
    results = [verify(c) for c in cands]
    if any(r.passed for r in results):
        return accept(select(results))
    feedback = describe_failures(results)   # e.g. "cand 2: 44 chars, budget 38; dropped '3'"
return escalate(cue, reason=feedback)       # cue left unchanged, reason recorded
```

Three rounds maximum. A timeout, rate limit, malformed JSON or empty response counts as a failed round. The loop can never ship a failing line, because acceptance only happens through `verify`.

10 EDGE CASES

Case	What goes wrong	How CueGate handles it
23.976 fps file	Float drift puts cues on the wrong frame	Rational frame rate, integer frames throughout
Subtitle authored at 25 fps, video at 23.976	Every rule is measured against the wrong clock	Compare video metadata with the declared rate; refuse to run on mismatch
BOM, CRLF, Latin-1 encoding	Parser errors or mojibake	Detect encoding, normalise before parsing
Malformed SRT (missing blank line, bad index)	Silent cue merge	Strict parse, report the exact line, no repair attempted
Overlapping cues from two speakers	False overlap violations	Profile switch for allowed overlaps; otherwise flagged
Speaker dashes in one cue	Line breaker or model merges speakers	Speaker lines are atomic; dash count is an invariant
Sentence continues into the next cue	Condensing breaks the bridge	Continuation flag; neighbours sent as context; bridging words protected
Italics and position tags	Tags counted as characters or lost	Measured stripped, preserved raw, balance checked
Emoji and combining accents	Code-point counts disagree with what viewers see	Count grapheme clusters
Song lyrics and music notes	Condensing lyrics is inappropriate	Flagged and skipped, never condensed
Forced narratives and on-screen text	Meaning tied to the picture	Flagged and skipped in v1
Single-word cue below minimum duration	Model asked to shorten "Hey!"	Timing-only path; never reaches the model
Cue already longer than 7 s	Condensing cannot fix duration	Resegmentation path
Zero or negative duration	Division by zero in CPS	Reported as a hard parse error
Last cue near video end	Ceiling computed past the end	Ceiling clamped to the final frame
Shot detector fires on a flash or fade	Phantom cuts distort timing rules	Adaptive detector with minimum scene length; cut-list import override
Dialogue genuinely crosses a cut	Rule would wrongly forbid it	Crossing allowed when the source already crossed
Model returns text over budget	Fails the gate	Rejected by the counter, feedback sent, next round
Model returns invalid JSON or nothing	Pipeline crash	Counted as a failed round
Subtitle text containing instructions	Prompt injection through data	Delimited, labelled as data, and output can only be accepted by the counter
Numbers written as words	"three" vs "3" flagged as a dropped number	Normalised before comparison
Names already established	Guideline allows dropping them	Allowed only if seen in the previous cues

Case	What goes wrong	How CueGate handles it
API down or no key	Whole run fails	`check` and timing repair run fully offline; condense steps escalate

11 EVALUATION PLAN

Corpus. English subtitles from Blender Foundation open movies, which are released under Creative Commons licences. Which films ship subtitle tracks, and under what licence, will be confirmed on day one before any file enters the repo.

Stress generator. Clean professional files rarely violate rules, so violations will be manufactured deterministically: scale cue durations by 0.6, 0.75 and 0.9 with a fixed seed. Because the original text is known, every shortened line has a ground truth to compare against.

Baselines.

Id	System	Purpose
B0	Word-boundary truncation to budget	The dumbest thing that fits
B1	Single model call, "shorten this to N characters", no gate	A naive baseline. It is not a reproduction of any product above.
B2	CueGate without the meaning guard	Isolates what the guard contributes
B3	Full CueGate	The system

Metrics.

- Violations per rule, before and after.
- Share of cues fixed by timing only, by line breaking, by resegmentation, by condensing, and escalated.
- Shipped violations. Must be zero by construction for B3; any non-zero value is a bug, not a result.
- Meaning-drift rate on the human audit, per system, with a 95% Wilson interval.
- Guard false-pass rate: shortened lines the guard accepted that the audit marked as changed in meaning. This is the honest headline number.
- Mean retained-token ratio, as a measure of minimal intervention.
- Model calls and tokens per film.

Human audit. 100 shortened cues, stratified across systems, shuffled and labelled blind by the owner as preserved, changed, or unclear. Labels and the labelling script will be in the repo.

Negative results. Every metric is reported whether it favours CueGate or not.

12 END TO END ON A MOVIE (PLANNED WALKTHROUGH)

> CONSTRUCTED. The commands below are the planned interface. No output shown here is real, and no numbers are given because none exist yet.

```
pip install -e ".[dev]"
cuegate check data/open/<film>.en.srt --video data/open/<film>.mp4 --profile en-adult
#  -> violation table by rule and cue; writes runs/<id>/report.json

cuegate fix data/open/<film>.en.srt --video data/open/<film>.mp4 --profile en-adult
#  -> runs/<id>/<film>.fixed.srt, report.json, report.html
#     every cue: action taken, checks passed, or escalation reason

cuegate eval --corpus data/open --stress 0.75 --systems B0,B1,B2,B3
#  -> runs/<id>/eval.html with every metric in section 11

cuegate serve
#  -> local web app at http://localhost:8000
```

13 WEB APP AND VISUAL DESIGN

The interface will use a dark, cinematic streaming-room look built from original design tokens. It will not use any streaming service's logo, wordmark, symbol or proprietary typeface, and nothing in it will suggest affiliation with any platform.

Token	Value	Use
`--bg`	#0A0A0A	Page
`--surface`	#141414	Panels
`--surface-2`	#1E1E1E	Raised panels, rows
`--line`	#2A2A2A	Dividers
`--text`	#F2F2F2	Primary text
`--muted`	#9A9A9A	Secondary text
`--accent`	#E3262F	Marquee red: failures, primary action
`--pass`	#2BD47D	Passing cues
`--warn`	#F2B01E	Warnings, escalations
Display font	Bebas Neue (SIL OFL)	Screen titles
UI font	Inter (SIL OFL)	Everything else
Mono font	JetBrains Mono (SIL OFL)	Timecodes, counts

Screens.

1. **Lobby.** A poster-style hero with a drop zone for a subtitle file and video. Below it, horizontal rows: "Open films" (the bundled corpus) and "Recent runs".
2. **Screening room.** The player with the live cue overlaid; under it a filmstrip timeline with white ticks for shot changes and cue blocks coloured by status; a violation rail on the right. Keys: J/K/L to scrub, N for next violation.
3. **Cut room.** Source line against chosen line, a budget meter ("31 / 38"), invariant chips, the NLI score, the other candidates, and Accept, Reject or Edit.

4. **Credits.** The report: before and after bars per rule, the escalation list with reasons, exports for SRT, JSON and HTML.

Status is never shown by colour alone: every state also carries a label and an icon.

14 COURSE SKILLS MAP

Course concept	Where it lives in CueGate	Evidence a reviewer can check
Prompt contracts	Condense call with budget, `must_keep`, strategies and a JSON schema	`cuegate/llm/prompts/condense_v1.md`, schema tests
Verification	Rule engine, invariants, NLI guard, fail-closed acceptance	Unit and property tests; zero shipped violations in eval
Agent loops	Bounded repair loop with structured failure feedback and escalation	Loop tests with a scripted fake model
Evaluation	Corpus, stress generator, four baselines, blind human audit	`cuegate eval` report and audit labels in the repo
Source grounding	Every rule and every violation cites its guideline section and read date	`source` field in every report line

15 SCOPE

In. English subtitles; SRT and WebVTT; adult and kids English profiles; shot detection or imported cut lists; timing, line-break, resegment and condense repairs; meaning guard; evaluation; local web app.

Out (deliberately). Translation. CueGate works within one language and will not translate. Also out for v1: SDH and caption-specific rules, TTML/IMSC export, right-to-left and CJK languages, burned-in subtitles, audio analysis.

First thing that will run. `cuegate check` alone: the rule engine on one open film's English subtitles, printing every violation. Someone will know it ran because its counts will match a fixture of cues checked by hand, and the test that compares them will pass in CI.

16 RISKS

Risk	Why it is specific to CueGate	Response
Lines pass every rule but change meaning	The central failure mode	Invariants, NLI, escalation, and a published false-pass rate from the audit
Shot detection errors	Fades and flashes create phantom cuts, so timing rules fire wrongly	Adaptive detector, minimum scene length, cut-list override, detector errors reported separately
Rule values wrong	Secondary sources disagree on CPS and counting convention	Values read from the primary guideline with a date; profiles editable
No subtitle tracks with usable licences	The corpus is the evaluation	Verify on day one; fallback is a self-written synthetic dialogue set, labelled as synthetic
Source out-time is not end of speech	v1 has no audio, so the timing solver may linger over silence or cut speech	Stated limitation; v2 adds voice activity detection
Model cost and latency	Calls per film could grow	Model called only after timing fails; caching; per-run call budget
Scope creep into translation or captions	Tempting and endless	Out list above; changes logged in FRICTIONAL.md

17 ROADMAP

Milestone	Delivers	Done when
M0 Film week	PROJECT-BRIEF, beat sheet, pitch film	Film passes Brutalist gates and is submitted
M1 Checker	IO, timebase, text metrics, rule engine, `cuegate check`	Golden fixture matches hand-checked counts; CI green
M2 Timing	Shot detection, cut rules, timing solver, line breaker, resegmenter	Timing-only fixes pass full re-check on stressed corpus
M3 Condense	Prompt contract, invariants, selection, bounded loop, escalation	Zero shipped violations; loop tests pass with fake model
M4 Evaluate	NLI guard, baselines, stress generator, human audit	Eval report generated with all section 11 metrics
M5 Ship	Web app, HTML report, docs, demo recording	A stranger can clone, install and run `cuegate eval` in under ten minutes

18 TODO

Before the film (day one).

- ☐ Re-open Subtitle Edit, Editsub, EZTitles and Ooona documentation; record exactly what each does, with the date.

- [ ] Search for any published meaning-drift or accuracy figure for AI subtitle shortening. If one exists, change the third differentiator.
- [ ] Read the Netflix English timed text style guide directly; confirm CPS limits, characters per line and the counting convention; record the read date.
- [ ] Confirm which Blender open movies ship English subtitle tracks and their licences.
- [ ] Check the name "CueGate" on GitHub and PyPI.

M1.

- [ ] `pyproject.toml`, ruff, mypy, pytest, GitHub Actions.
- [ ] `FrameRate`, ms to frame conversion, round-trip property tests.
- [ ] SRT and WebVTT parsing with strict error reporting.
- [ ] Grapheme counting and tag stripping.
- [ ] Profile loader with schema validation.
- [ ] Rules `CPS_MAX`, `CPL_MAX`, `LINES_MAX`, `DUR_MIN`, `DUR_MAX`, `GAP_MIN`, `OVERLAP`.
- [ ] `cuegate check` with table output and `report.json`.
- [ ] Hand-checked golden fixture.

M2.

- [ ] Shot detection with cache; cut-list import.
- [ ] `CUT_CROSS`, `CUT_IN_SNAP`, `CUT_OUT_SNAP`.
- [ ] Timing solver with property tests (never creates a new violation).
- [ ] Line breaker and resegmenter.

M3.

- [ ] Claude client: pinned model, JSON schema, retries, cache, dated call log.
- [ ] `condense_v1` prompt and schema tests.
- [ ] Invariants, selection, bounded loop, escalation.
- [ ] Fake-model test harness for the loop.

M4.

- [ ] NLI guard and threshold calibration.
- [ ] Stress generator and baselines B0 to B3.
- [ ] Blind audit tool and labels.
- [ ] Eval report.

M5.

- [ ] FastAPI endpoints; React app screens.
- [ ] HTML report; README quickstart; demo recording.

19 WHY THIS DESIGN, AND WHERE IT IS NOT STATE OF THE ART

API models cannot be forced to hit an exact character count during decoding, so generate, verify and repair with a deterministic checker is the practical pattern for hard constraints. CueGate applies that pattern carefully; it does not invent it. The NLI check is a known-imperfect proxy for meaning, which is why it is called the weaker layer and why its error rate is measured rather than assumed. The timing solver is only as good as the source timing, because v1 does not listen to the audio. The line breaker uses word lists, not a parser. Each of these is a place a better system would do more, and each is stated here so the pitch does not overclaim.

20 REPOSITORY LAYOUT

```
cuegate/
  cuegate/
    io/ timebase.py text.py
    shots/ rules/ profiles/
    repair/ guard/ llm/ eval/ api/
    cli.py
  web/
  tests/
    fixtures/  golden/  property/
  data/open/        # licensed corpus only, with LICENSES.md
  runs/             # ignored by git
  docs/
  FRICTIONAL.md
  README.md
```

21 GETTING STARTED (PLANNED)

> CONSTRUCTED. None of this works yet.

```
git clone https://github.com/DeepakKTS/cuegate
cd cuegate
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
cp .env.example .env          # ANTHROPIC_API_KEY, only needed for condensing
pytest
cuegate check tests/fixtures/sample.en.srt --fps 24 --profile en-adult
```

22 EXISTS TODAY, AND WILL EXIST

Exists today (2026-10-14).

- ■ This planning document.
- ■ A draft rule profile with values taken from secondary sources, not yet confirmed against the primary guideline.

Will exist.

- The `cuegate` package, CLI and web app.
- The rule engine, timing solver, line breaker, resegmenter, condenser, meaning guard and bounded loop.
- The licensed corpus, stress generator, baselines, blind human audit and evaluation report.

23 REFERENCES

All pages read 2026-10-14 unless marked otherwise.

- Netflix Partner Help, Timed Text Style Guide: Subtitle Timing Guidelines. https://partnerhelp.netflixstudios.com/hc/en-us/articles/360051554394
- Netflix Partner Help, Timed Text Style Guide: Subtitle Templates. https://partnerhelp.netflixstudios.com/hc/en-us/articles/219375728
- Netflix Partner Help, Timed Text Style Guide: General Requirements. https://partnerhelp.netflixstudios.com/hc/en-us/articles/215758617
- Gotham Lab, Netflix Subtitle Delivery Requirements (secondary). https://www.gothamlab.com/netflix-subtitle-delivery-requirements-complete-guide/
- vSubtitle, Netflix Subtitle Guidelines Explained (secondary). https://vsubtitle.com/netflix-subtitle-guidelines-explained/
- Subhero, Subtitle Standards Guide (the source of the dropped claim). https://subhero.io/blog/subtitle-standards-guide
- Subtitle Edit documentation. https://subtitleedit.github.io/subtitleedit/
- Editsub. https://editsub.com/
- EZTitles. https://eztitles.com/ and https://eztitles.com/Webhelp/EZTitles/subtitles_editing.htm
- Subtitle AI. https://www.subtitle.ai/
- Smartcat, Review and edit AI translations in the Subtitle Editor. https://help.smartcat.com/review-edit-subtitle-translations/
- Brutalist video toolkit. https://github.com/nikbearbrown/brutalist.art